**ШИНЖЛЭХ УХААН, ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ**
**МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ**




# БИЕ ДААЛТЫН ТАЙЛАН

**Үйлдлийн систем (F.CSM302): V-1-12**

| | |
|---|---|
| **Төслийн нэр:** | The Producer–Consumer Problem (Programming Project 4) — Buffer animation |
| **Шалгасан багш:** | Г.Ганбат /F.CS10/ |
| **Бие даалтын ажил гүйцэтгэсэн:** | B241910012 Н. Баасандорж<br>B241910035 Э. Анунбаяр<br>B222230020 .....<br>B242270014 .....<br>B232270028 ..... |

**Улаанбаатар хот, 2026.**

## Гарчиг:

**Хураангуй** ..... 3
**1. Удиртгал** ..... 3
1.1 Зорилго ..... 3
1.2 Ач холбогдол ..... 3
1.3 Хамрах хүрээ ..... 3
**2. Сурах бичгийн шаардлага** ..... 4
2.1 Шаардлагуудын хэрэгжилт ..... 4
**3. Онолын үндэс** ..... 4
3.1 Thread, race condition ба синхрончлол ..... 4
3.2 Алгоритм: Bounded-buffer ..... 5
3.3 Өгөгдлийн бүтэц ..... 6
**4. Системийн зохиомж** ..... 6
**5. Хэрэгжилт** ..... 8
5.1 Хэл, сан, орчин ..... 8
5.2 Тойрог буфер ..... 8
5.3 Producer ба consumer thread ..... 9
5.4 Синхрончлол ..... 9
5.5 UI удирдлага ба C–UI холбоо ..... 9
5.6 Алдааны туршилтын горим ..... 10
5.7 Репозиторийн бүтэц ..... 10
**6. Туршилт ба UI дүрслэл** ..... 10
6.1 Юуг өөрчилж туршиж болох вэ ..... 10
6.2 Юуг ажиглах вэ ..... 11
6.3 UI-ийн дэлгэцийн зураг ..... 11
**7. Туршилтын арга** ..... 14
7.1 Туршилтын орчин ..... 14
7.2 Туршилтын өгөгдөл ..... 14
7.3 Хүлээгдэж буй үр дүн ..... 15
**8. Үр дүн** ..... 15
8.1 Хэвийн ба хязгаарын тестийн үр дүн ..... 16
8.2 Алдааны горимын туршилтын үр дүн ..... 16
8.3 Буруу оролтын тестийн үр дүн ..... 18
8.4 Автомат тестийн үр дүн ..... 19
8.5 Хэрэглэгчийн интерфейсийн (UI) тестийн үр дүн ..... 19
8.6 Үр дүнгийн тайлбар ..... 20
**9. Хэлэлцүүлэг** ..... 20
9.1 Үр дүнгийн тайлбар ..... 20
9.2 Хязгаарлалт ..... 21
**10. Дүгнэлт** ..... 21
**11. Багийн гишүүдийн оролцоо** ..... 22
**Ашигласан материал** ..... 22

# Хураангуй

**Асуудал:** Энэхүү төслийн зорилго нь F.CSM302 Үйлдлийн систем хичээлийн 7-р бүлэг (Synchronization Examples)-ийн Programming Project 4 буюу Producer–Consumer (bounded-buffer) бодлогыг C хэл дээр POSIX Pthreads ашиглан хэрэгжүүлж, олон thread-ийн синхрончлолыг буферийн animation бүхий интерактив веб интерфейсээр харуулах юм.

**Шийдэл:** Хязгаартай тойрог буферийг `empty`, `full` тоолох семафор болон `pthread_mutex_t` mutex-ээр хамгаалсан. Producer thread-үүд санамсаргүй тоо үүсгэж `insert_item()`-ээр буферт тавьж, consumer thread-үүд `remove_item()`-ээр буферээс авна. C програм `--ui` горимд thread бүрийн төлөв, семафорын утга, mutex-ийн эзэмшигчийг stdout-оор гаргаж, Python стандарт сангийн веб сервер (`gui.py`) үүнийг хөтөч рүү дамжуулж, хөтөч дээр buffer animation болгон харуулна. Нэмэлтээр Python (`threading`) хэрэгжүүлэлттэй, монгол дуут тайлбартай «шуудангийн хайрцаг» жишээтэй горим бий.

**Туршилт:** C програмын командын мөрийн туршилт (`make test`): хэвийн, хязгаарын болон буруу оролтын тохиолдлууд; алдааны дөрвөн горим (mutex-гүй, `wait(empty)`-гүй, `wait(full)`-гүй, lock–wait буруу дараалал); Python `unittest`-ийн 42 тест; ThreadSanitizer. Веб UI-ийн удирдлагыг HTTP түвшинд автоматаар, харагдах байдлыг дэлгэцийн зургаар шалгасан.

**Үр дүн:** Зөв кодын бүх ажиллагаанд `produced − consumed = in_buffer` нөхцөл биелж `CONSISTENT` гарсан. Алдааны горим бүрийг програм илрүүлж `INCONSISTENT` (exit 2) эсвэл `DEADLOCK` (exit 3) гаргасан. 42 автомат тестээс 41 нь амжилттай, 1 нь зориудаар алгассан (Python хөдөлгүүрт mutex-ийн унтраалга байхгүй). ThreadSanitizer анхааруулга гаргаагүй.

# 1. Удиртгал

## 1.1 Зорилго

Төслийн зорилго нь олон thread зэрэг ажиллах үед хуваалцсан өгөгдөл (bounded buffer)-д хандах хандалтыг семафор ба mutex-ээр хэрхэн зөв синхрончилдгийг бодит C програмаар хэрэгжүүлж, thread бүрийн хүлээлт, mutex-ийн эзэмшил, буферийн дүүрэх ба хоосрох үйл явцыг animation-аар харуулах явдал юм. Мөн синхрончлолын нэг алдаа гарвал (race condition, overflow, underflow, deadlock) юу болохыг туршиж харуулна.

## 1.2 Ач холбогдол

Producer–Consumer нь үйлдлийн системийн синхрончлолын сонгодог бодлого бөгөөд pipe, мессежийн дараалал, хэвлэх дараалал (print spooler) зэрэгт ижил зарчим ажилладаг. Race condition нь ховор, давтагдахгүй байдлаар гарч, ажиглахад хэцүү байдаг. Иймд thread бүрийн төлөв, семафорын бодит утга, mutex-ийн эзэмшигчийг харагдахуйц болгож, зориудаар алдаа гаргаж туршсанаар `wait/signal`-ийн үүрэг ба тэдгээрийн дарааллын ач холбогдлыг ойлгоход энэ ажил тусална.

## 1.3 Хамрах хүрээ

- Хязгаартай тойрог буфер: `insert_item()`, `remove_item()`, BUFFER_SIZE 1–20.

- `empty`, `full` тоолох семафор болон `mutex`-ээр хамгаалах синхрончлол.
- Producer, consumer thread-ийн тоо, унтах хугацаа, хурдыг тохируулах.
- Start, Pause, Step, Stop, Reset удирдлага ба ажиллаж байх үеийн тохиргоо.
- Алдааны дөрвөн горим ба тэдгээрийг илрүүлэх шалгагч.
- Эцсийн тууштай байдлын шалгалт: `produced − consumed = in_buffer`.
- Хамаарахгүй зүйл: олон процесс, сүлжээний хэрэгжүүлэлт, Windows дээр C хөдөлгүүрийг шууд ажиллуулах (WSL2 шаардлагатай).

# 2. Сурах бичгийн шаардлага

Төсөл нь Silberschatz, Galvin, Gagne нарын «Operating System Concepts» (10-р хэвлэл) сурах бичгийн 7-р бүлгийн 7.1.1 (The Bounded-Buffer Problem) хэсэг болон бүлгийн төгсгөлийн Programming Project 4 (The Producer–Consumer Problem)-той холбоотой. Номд хуваалцсан буфер ашигладаг producer, consumer thread-үүдийг Pthreads, семафор, mutex lock ашиглан хэрэгжүүлэхийг шаарддаг. Бие даалтын нэмэлт шаардлага нь buffer animation бүхий хэрэглэгчийн интерфейс юм.

## 2.1 Шаардлагуудын хэрэгжилт

*Хүснэгт 2.1. Шаардлага ба төслийн шийдэл*

| Шаардлага: | Төслийн шийдэл: |
|---|---|
| Командын мөрийн аргумент: унтах хугацаа, producer тоо, consumer тоо | `./producer_consumer <sleep> <producers> <consumers> [max_nap]`; `parse_long` / `parse_double`-оор шалгаж, буруу бол `usage` хэвлээд exit 1 |
| Хязгаартай буфер, BUFFER_SIZE = 5 | `buffer.h` / `buffer.c`: тойрог дараалал; `make BUFFER_SIZE=N`-ээр өөрчилнө; UI дээр 1–20 |
| `insert_item()`, `remove_item()`: амжилттай бол 0, алдаа бол -1 | Яг ийм буцаах утгатай; зогсоох командын үед thread цэвэрхэн гарахын тулд -1 буцаана |
| Producer: санамсаргүй хугацаа унтаж, санамсаргүй тоо үүсгээд буферт тавина | `main.c`: `producer()` — `nap()`, `rand_r()`, `insert_item()` |
| Consumer: санамсаргүй хугацаа унтаж, буферээс авна | `main.c`: `consumer()` — `nap()`, `remove_item()` |
| Синхрончлол: `empty`, `full` семафор ба `mutex` | `sem_t empty, full`; `pthread_mutex_t mutex` |
| Pthreads: thread үүсгэх, дуусахыг хүлээх | `pthread_create`, `pthread_join` |
| Buffer animation бүхий UI (бие даалтын шаардлага) | Веб UI: `raw.html` (C · Pthreads), `mail.html` (Python · шуудангийн жишээ); `gui.py` сервер |

# 3. Онолын үндэс

## 3.1 Thread, race condition ба синхрончлол

Thread нь процессийн дотор ажиллах гүйцэтгэлийн нэгж бөгөөд нэг процессийн thread-үүд глобал өгөгдөл болон heap санах ойг хуваалцдаг боловч тус бүр өөрийн stack,

регистртэй байна. Хоёр thread хуваалцсан өгөгдөлд зэрэг хандаж, үр дүн нь гүйцэтгэлийн дарааллаас хамаарч байвал **race condition** үүснэ. Жишээ нь `count++` нь «уншиж, нэмж, буцааж бичих» гурван машин команд тул producer `count++`, consumer `count--` зэрэг хийхэд нэг өөрчлөлт алдагдаж болно.

Хуваалцсан өгөгдөлд хандах кодын хэсгийг **critical section** гэнэ. Нэг цагт зөвхөн нэг thread түүнд орох ёстой. Үүнийг хангахад **mutex lock** (`pthread_mutex_lock/unlock`) ашиглана. **Семафор** нь бүхэл тоон хувьсагч S бөгөөд атомар хоёр үйлдэлтэй: `wait(S)` нь S ≤ 0 бол хүлээж, үгүй бол S-ийг 1-ээр бууруулна; `signal(S)` нь S-ийг 1-ээр нэмнэ. Тоолох семафор нь чөлөөтэй нөөцийн тоог төлөөлдөг. POSIX-д `sem_init`, `sem_wait`, `sem_post` үүнийг хэрэгжүүлнэ.

**Deadlock** үүсэхэд дөрвөн нөхцөл (mutual exclusion, hold and wait, no preemption, circular wait) зэрэг биелэх ёстой. Төслийн «lock → wait дараалал» алдааны горим нь «hold and wait»-ийг харуулна: thread mutex барьсаар семафор дээр хүлээнэ.

## 3.2 Алгоритм: Bounded-buffer

Сурах бичгийн (Figure 7.1, 7.2) алгоритмд гурван хэрэгслийг ашиглана: `empty = N` (хоосон нүдний тоо), `full = 0` (дүүрэн нүдний тоо), `mutex = 1`.

```
/* producer */                      /* consumer */
while (true) {                     while (true) {
    wait(empty);                       wait(full);
    wait(mutex);                       wait(mutex);
    /* буферт item нэмнэ */            /* буферээс item авна */
    signal(mutex);                     signal(mutex);
    signal(full);                      signal(empty);
}                                  }
```

Дараалал чухал: `wait(empty)` нь `wait(mutex)`-ээс **өмнө** байх ёстой. Буфер дүүрэн үед mutex-ийг эхэлж авсан producer `wait(empty)` дээр хүлээвэл consumer mutex авч чадахгүй тул хэн ч урагшлахгүй (deadlock). Буферийг тойрог дараалал болгон `in = (in + 1) mod N`, `out = (out + 1) mod N` индексээр удирдана. `insert_item`, `remove_item` бүр O(1) хугацаатай, буферийн санах ой O(N).

Алгоритмын тогтмол нөхцөлүүд (invariant):

1. `0 ≤ count ≤ N` — буферт хэзээ ч N-ээс олон item байхгүй, хоосон буферээс унших явдал гарахгүй.
2. `empty + full = N` — thread `sem_wait`-ийн дараа, `sem_post`-оос өмнө байх агшинд түр зуур 1–2-оор бага харагдаж болно.
3. `produced − consumed = count` — програмын эцсийн `CONSISTENT` шалгалтын үндэс.

Програмын ажиллагаа:

1. `main()` командын мөрийн аргументыг шалгана.
2. `buffer_init()` нь mutex-ийг, `empty = N`, `full = 0` семафорыг үүсгэнэ.
3. Producer болон consumer thread-үүдийг `pthread_create`-ээр үүсгэнэ.
4. Thread бүр санамсаргүй хугацаа унтаад `insert_item()` / `remove_item()` дуудна.

5. Заасан хугацааны дараа `running = 0` болгож, бүх thread-ийг `pthread_join`-оор хүлээнэ.
6. `produced − consumed == буферт үлдсэн`, `lost == 0`, `bad_reads == 0` ба deadlock байхгүй эсэхийг шалгаж `CONSISTENT`, `INCONSISTENT` эсвэл `DEADLOCK` гэж хэвлэнэ.

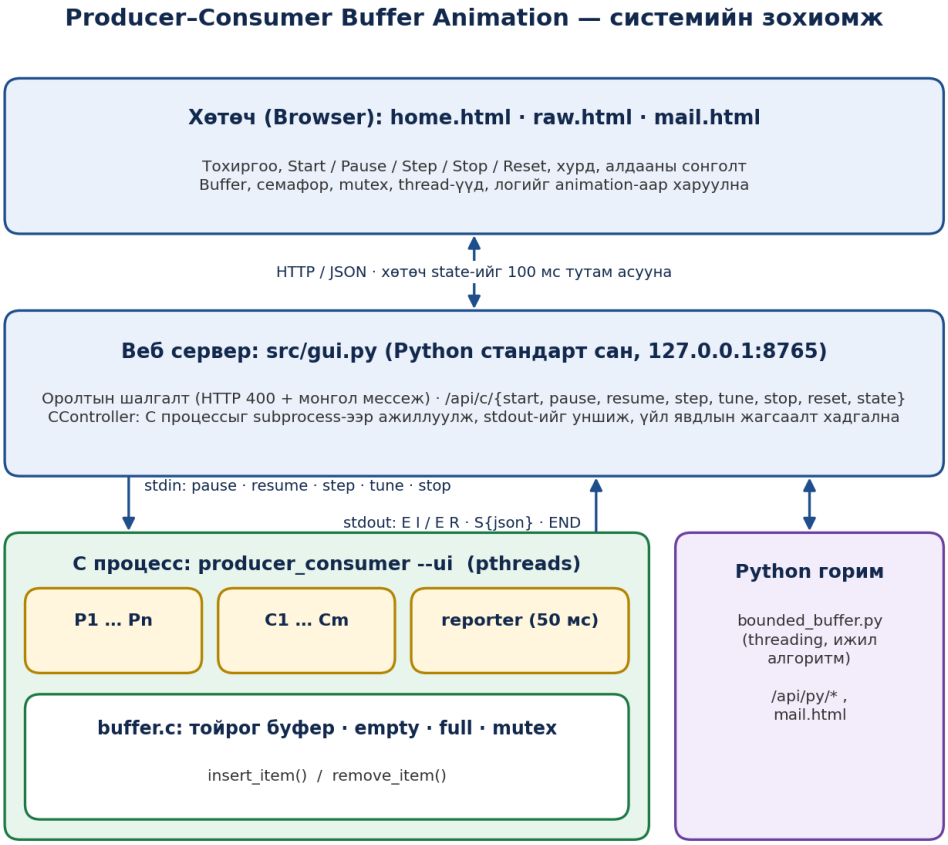
### 3.3 Өгөгдлийн бүтэц

*Хүснэгт 3.1. Ашигласан өгөгдлийн бүтцүүд*

| Бүтэц: | Хэрэглээ: |
|---|---|
| `buffer_item buffer[BUFFER_MAX]` | Тойрог буфер (BUFFER_MAX = 20); `buffer_item` нь `int` |
| `in`, `out`, `count`, `last` | Тойрог дарааллын индекс, item-ийн тоо, сүүлд хандсан нүд |
| `sem_t empty`, `sem_t full` | Хоосон ба дүүрэн нүдний тоолох семафор |
| `pthread_mutex_t mutex` | Буфер ба индексийг хамгаалах цоож |
| `atomic_int running`, `atomic_long produced, consumed, lost, bad_reads` | Зогсоох тугийн болон тоолуурууд; `produced`, `consumed`-ийг critical section дотор нэмнэ |
| `atomic_int occ[BUFFER_MAX]` | Нүд бүрт item байгаа эсэхийг тэмдэглэх шалгагч. Алгоритмын хэсэг биш, зөвхөн алдаа илрүүлэх хэмжилт |
| `_Thread_local char me[16]` | Thread-ийн нэр (P1, C2 …) — логонд хэрэглэнэ |
| `enum` thread төлөв | idle, sleeping, producing, waiting, consuming, paused, stopped |
| JSON snapshot ба үйл явдлын жагсаалт | C → UI мэдээлэл; `gui.py` хамгийн ихдээ 5000 үйл явдал хадгална |

## 4. Системийн зохиомж

Систем нь дөрвөн хэсэгтэй: (1) хөтөч дээрх хэрэглэгчийн интерфейс, (2) localhost веб сервер `gui.py`, (3) синхрончлолыг бодитоор гүйцэтгэдэг C процесс, (4) жишээтэй горимын Python хөдөлгүүр. Зохиомжийн гол зарчим: **бүх синхрончлол C програм дотор явагдана**. Хөтөч нь JavaScript дээр дуурайлган тооцоолохгүй, C процессоос ирсэн бодит үйл явдал ба төлвийг зөвхөн харуулж, удирдах команд илгээнэ.




*Зураг 1. Системийн зохиомж*

**Хөтөч:** `home.html` (горим сонгох), `raw.html` (C · Pthreads хуудас), `mail.html` (Python · шуудангийн жишээ). **Сервер `gui.py`:** Python стандарт сангийн `ThreadingHTTPServer`, зөвхөн `127.0.0.1` дээр сонсоно; `/api/c/*`, `/api/py/*` хаягаар `start, pause, resume, step, tune, stop, reset, state` үйлдлийг хүлээн авч, оролтыг шалгаад алдаатай бол HTTP 400 ба монгол мессеж буцаана. `CController` нь C процессыг `subprocess`-ээр ажиллуулж, stdout-ийг тусдаа thread-ээр уншиж, командыг stdin-ээр илгээнэ.

Өгөгдлийн урсгал:

1. Хэрэглэгч тохиргоогоо оруулж **Start** дарахад хөтөч `POST /api/c/start` илгээнэ.
2. `gui.py` параметрийг шалгана. Буруу бол 400 ба мессеж буцаана, зөв бол C процессыг `producer_consumer --ui <size> <producers> <consumers>` … аргументтайгаар ажиллуулна.
3. C процесс thread-үүдээ үүсгэнэ. Item бүр буферт орох/гарах бүрд `E I` … / `E R` … мөр, мөн 50 мс тутам `S {json}` төлвийн мөр stdout-д гарна.
4. `gui.py` мөр бүрийг задлан дугаартай (`seq`) үйл явдлын жагсаалтад хадгална.
5. Хөтөч 100 мс тутам `GET /api/c/state?since=<seq>` асууж, шинэ үйл явдлуудыг buffer дээрх нислэг, заагч (`in`, `out`), семафор, mutex-ийн animation болгон харуулна.
6. **Pause / Step / Resume / Stop / хурд** нь `POST` → stdin команд (`pause`, `step`, `resume`, `stop`, `tune` …) болж C процесст хүрнэ.

7. **Stop** дарахад thread-үүд `join` хийгдэж, эцсийн төлөв ба `END` гарна. Лог дээр `produced=… consumed=… in_buffer=…` → `CONSISTENT / INCONSISTENT / DEADLOCK` гарна.

Python горим (`bounded_buffer.py`) нь ижил алгоритмыг `threading.Semaphore`, `threading.Lock`-оор хэрэгжүүлж, «шуудан зөөгч (producer) – шуудангийн хайрцаг (buffer) – хэрэглэгч (consumer)» жишээгээр монгол дуут тайлбартай ойлгомжтой тайлбарладаг.

**Дүгнэлт:** С процесс, сервер, хөтөчийн үүргийг тусгаарласнаар синхрончлолын бодит ажиллагааг (thread, семафор, mutex) өөрчлөхгүйгээр харагдуулах боломж бүрдсэн.

# 5. Хэрэгжилт

## 5.1 Хэл, сан, орчин

- **C (C11) + POSIX Pthreads** — `pthread`, `sem_t`, `<stdatomic.h>`; хөрвүүлэлт `gcc -Wall -Wextra -O2 -std=c11 -pthread`.
- **Python 3 (зөвхөн стандарт сан)** — `gui.py` веб сервер, `bounded_buffer.py`, `unittest`.
- **HTML / CSS / JavaScript** — framework-гүй, гадаад сангүй UI.
- **GNU Make** — `make`, `make test`, `make tsan`.
- Гадаад сан ашиглаагүй. Дуут тайлбарын аудио (сонголттой) `tools/make_voice.py` (edge-tts)-оор үүснэ.

## 5.2 Тойрог буфер

Буфер нь `buffer_item buffer[BUFFER_MAX]` массив ба `in`, `out`, `count` индекстэй. Буферийн хэмжээг `buffer_init(n)` ажиллах үед тогтооно (n ≤ 0 бол BUFFER_SIZE = 5). Доор `insert_item()`-ийн зөв кодын хэсгийг хураангуйлж үзүүлэв (алдааны горимын салаа хасагдсан).

```
int insert_item(buffer_item item) {
    if (wait_or_stop(&empty) != 0) return -1;  /* wait(empty)  */
    if (lock() != 0)               return -1;  /* wait(mutex)  */
    int slot = in;                              /* critical section */
    buffer[slot] = item;
    in = (slot + 1) % size;
    count = count + 1;
    sim_on_insert(item, slot, count, 0);        /* лог, mutex дотор */
    unlock();                                   /* signal(mutex) */
    sem_post(&full);                            /* signal(full)  */
    return 0;
}
```

`remove_item()` нь тэгш хэмтэй: `wait(full)` → `lock` → `item = buffer[out]; out = (out + 1) % size; count--` → `unlock` → `sem_post(&empty)`. Зөв код дээр `count >= size` (insert) эсвэл `count <= 0` (remove) нөхцөл хэзээ ч биелэхгүй; хэрэв биелвэл item-ийг буцааж, семафорыг сэргээж `-1` буцаана.

### 5.3 Producer ба consumer thread

Producer thread нь `while (running)` давталтдаа `[0, max_nap]` секундийн санамсаргүй хугацаа унтаж (`nap()`), `rand_r()`-ээр item үүсгээд `insert_item()` дууддаг. `rand_r()` нь thread бүр өөрийн `seed`-тэй тул thread-safe. Consumer thread мөн адил унтаж `remove_item()` дууддаг. `main()` нь thread-үүдийг үүсгээд заасан хугацаанд унтаж, дараа нь `running = 0` болгож `pthread_join` хийгээд эцсийн шалгалтыг гүйцэтгэнэ. Гарах код: **0** — CONSISTENT, **1** — буруу оролт, **2** — INCONSISTENT, **3** — DEADLOCK.

### 5.4 Синхрончлол

Гурван хэрэгсэл (`empty`, `full`, `mutex`) нь сурах бичгийн дарааллаар ашиглагдана. Нэг ялгаа бий: энгийн `sem_wait` / `pthread_mutex_lock` хүлээж байгаа thread-ийг зогсоох боломжгүй тул программ `sem_timedwait` ба `pthread_mutex_timedlock`-ийг 50 мс-ийн хугацаатай давтаж, `running` тугийг шалгадаг (`wait_or_stop()`, `lock()`). Ингэснээр Stop дарахад thread-үүд хүлээлтээсээ гарч цэвэрхэн дуусдаг. Энэ нь deadlock-ийн туршилтад онцгой чухал: түгжигдсэн thread-үүдийг ч зогсоож, `join` хийж чаддаг.

### 5.5 UI удирдлага ба C–UI холбоо

`ui.c` нь `--ui` горимд ижил thread, семафор, mutex-ийг ашиглан дараах нэмэлтийг хийдэг:

- **Pause / Step / Resume:** thread бүр үйлдлийн өмнө `checkpoint()`-д ирж, `paused` үед хүлээнэ. **Step** нь `step_sem` семафорыг нэмж нэг thread-ийг нэг алхмаар урагшлуулна.
- **Хурд ба унтах хугацаа ажиллах үед өөрчлөгдөх:** `sim_sleep()` нь 50 мс-ээр хуваан унтаж, унтах хугацаа ба хурдыг mutex-ээр хамгаалагдсан хувьсагчаас дахин уншдаг тул `tune` команд тэр даруй хүчинтэй болно.
- **Төлөв:** `reporter` thread 50 мс тутам JSON snapshot (буферийн нүд, `in`/`out`, `empty`/`full`, mutex эзэмшигч, thread бүрийн төлөв ба хүлээж буй зүйл, тоолуур) хэвлэнэ.
- **Харагдах хугацаа:** «producing», «consuming» төлөв 0.3 с (HOLD) харагдана — зөвхөн дүрслэлийн сунгалт.

Оролтын хүрээ (UI): BUFFER_SIZE 1–20, producers 1–10, consumers 1–10, унтах хугацаа 0–3 с (алхам 0.05), хурд 0.25–4×. C хөдөлгүүр өөрөө мөн адил шалгаж, хүрээнээс гарсан утгыг хүлээж авахгүй.

### 5.6 Алдааны туршилтын горим

Зөв кодоос нэг сонгодог алдаа гаргаж, юу болохыг харуулдаг горим (CLI: `--bug`, UI: «алдаа» сонголт). Нүд бүрт item байгаа эсэхийг `occ[]` атомар шалгагчаар хянаж, алдаа бүрийг илрүүлдэг.

*Хүснэгт 5.1. Алдааны горимууд*

| Горим: | Кодын алдаа: | Үр дагавар: |
|---|---|---|
| `no-mutex` | `pthread_mutex_lock/unlock` байхгүй | Race condition: хоёр thread нэг `in` утгыг уншиж нэг нүд рүү бичнэ. Item алга болно (lost), хоосон нүднээс уншина (garbage) |
| `no-empty` | Producer `sem_wait(&empty)` хийхгүй | Overflow: буфер дүүрсэн ч бичсээр, уншаагүй item дээр дарж бичнэ |
| `no-full` | Consumer `sem_wait(&full)` хийхгүй | Underflow: хоосон нүднээс уншина, тоолуур сөрөг болно |
| `order` | `mutex_lock`-ийг `sem_wait`-аас өмнө хийнэ | Deadlock: mutex барьсан thread семафор хүлээж, бусад нь mutex хүлээнэ. Хэн ч урагшлахгүй |

`no-mutex` горимд race condition-ийг харагдахуйц болгохын тулд critical section дотор богино завсар (UI: 40 мс, CLI: 0.1 мс) нэмсэн. Энэ нь тэр мөчид өөр thread тасалдуулсныг (preemption) дуурайлгана. Бусад горимд завсар байхгүй.

### 5.7 Репозиторийн бүтэц

*Хүснэгт 5.2. Файл, хавтас*

| Файл/хавтас: | Зориулалт: |
|---|---|
| `src/c/buffer.c`, `buffer.h` | `insert_item`, `remove_item`, `empty`, `full`, `mutex`, алдааны горим |
| `src/c/main.c` | Номын дагуух CLI: аргумент шалгах, thread үүсгэх, эцсийн шалгалт |
| `src/c/ui.c`, `sim.h` | Вебт төлөв дамжуулах `--ui` горим (pause/step/tune, snapshot) |
| `src/c/Makefile` | `make`, `make test`, `make tsan` |
| `src/gui.py` | localhost веб сервер ба API; C процессыг удирдана |
| `src/bounded_buffer.py` | Python хэрэгжүүлэлт (шуудангийн жишээтэй горим) |
| `src/ui/` | `home.html`, `raw.html`, `mail.html`, `tour.json`, `voice/` |
| `tests/` | `test_buffer.py`, `test_c.py`, `test_web.py` (нийт 42 автомат тест) |
| `docs/V-1-12_Manual.pdf` | Гарын авлага |
| `tools/make_voice.py` | Дуут тайлбарын аудио үүсгэгч |
| `README.md` | Төслийн тайлбар, ажиллуулах заавар |

## 6. Туршилт ба UI дүрслэл

### 6.1 Юуг өөрчилж туршиж болох вэ

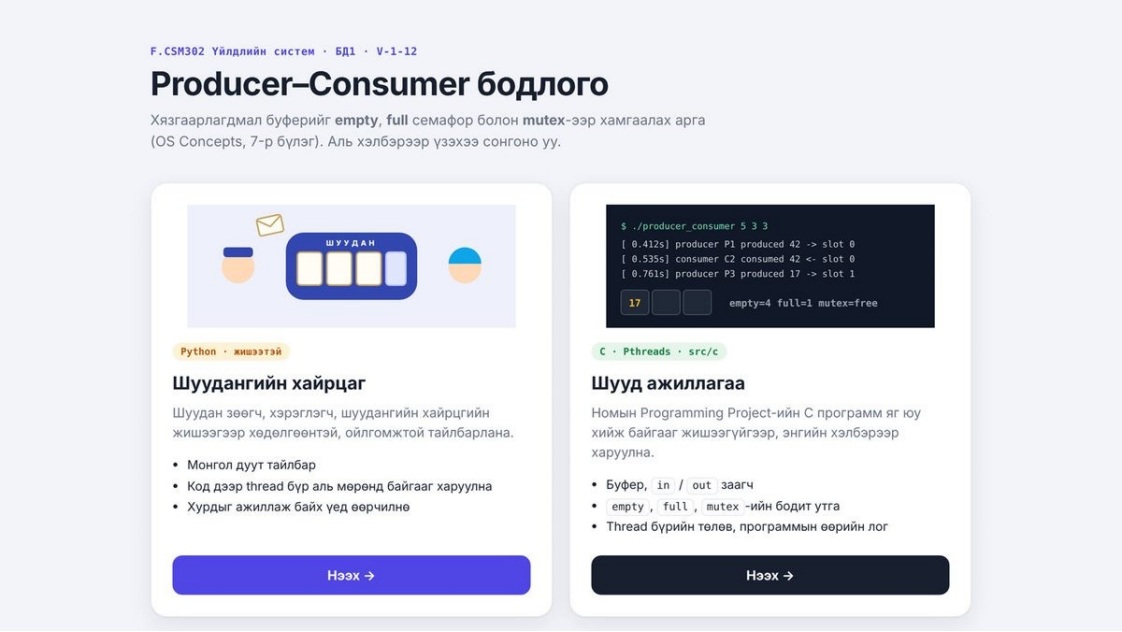
- BUFFER_SIZE (1–20), producer ба consumer thread-ийн тоо (1–10).

- Producer болон consumer-ийн унтах хугацаа (0–3 с) ба хурд (0.25–4×) — ажиллаж байх үед ч.
- Start, Pause, Step, Resume, Stop, Reset үйлдлүүд.
- Алдааны горим: зөв код, mutex-гүй, `wait(empty)`-гүй, `wait(full)`-гүй, lock → wait дараалал.
- Буруу оролт: хүрээнээс гарсан утга, тоо биш утга.

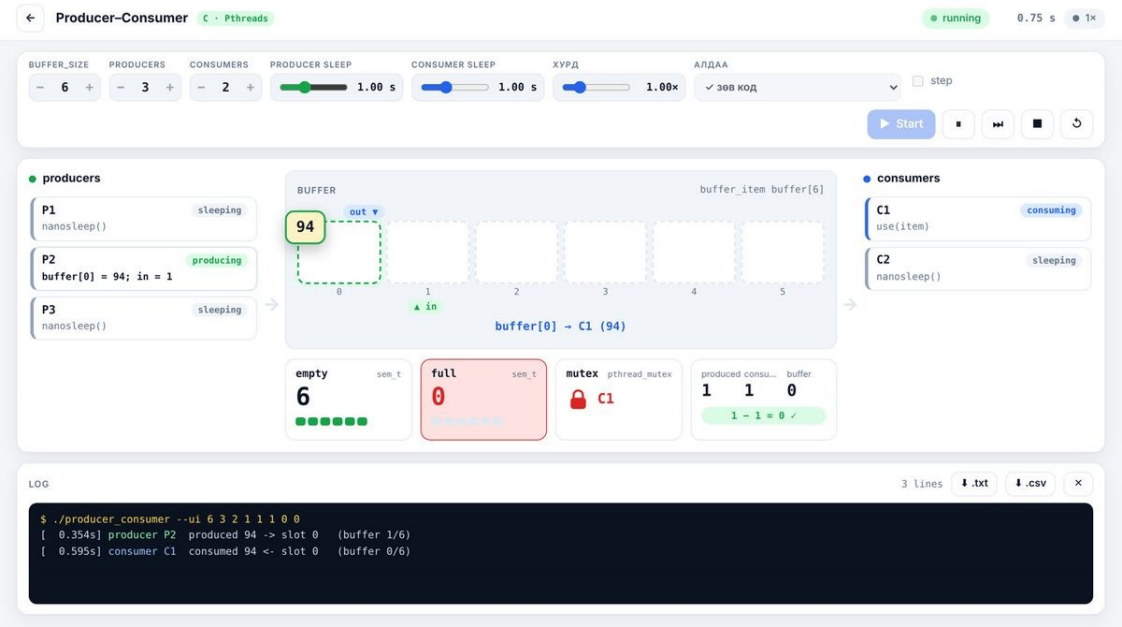
### 6.2 Юуг ажиглах вэ

- Producer-ууд хурдан үед буфер дүүрч `empty = 0` болж, producer-ууд `sem_wait(&empty)` дээр хүлээж байгаа эсэх.
- Consumer-ууд хурдан үед буфер хоосорч `full = 0` болж, consumer-ууд `sem_wait(&full)` дээр хүлээж байгаа эсэх.
- Mutex-ийг нэг дор зөвхөн нэг thread эзэмшиж байгаа эсэх, бусад нь хүлээж буй эсэх.
- `produced − consumed = buffer` шалгалт ногоон хэвээр байгаа эсэх.
- Алдааны горимд алдаа улаанаар тэмдэглэгдэж, лог ба эцсийн дүнд гарч байгаа эсэх.
- Pause / Step / Resume / Reset үед thread-ийн төлөв зөв өөрчлөгдөж байгаа эсэх.

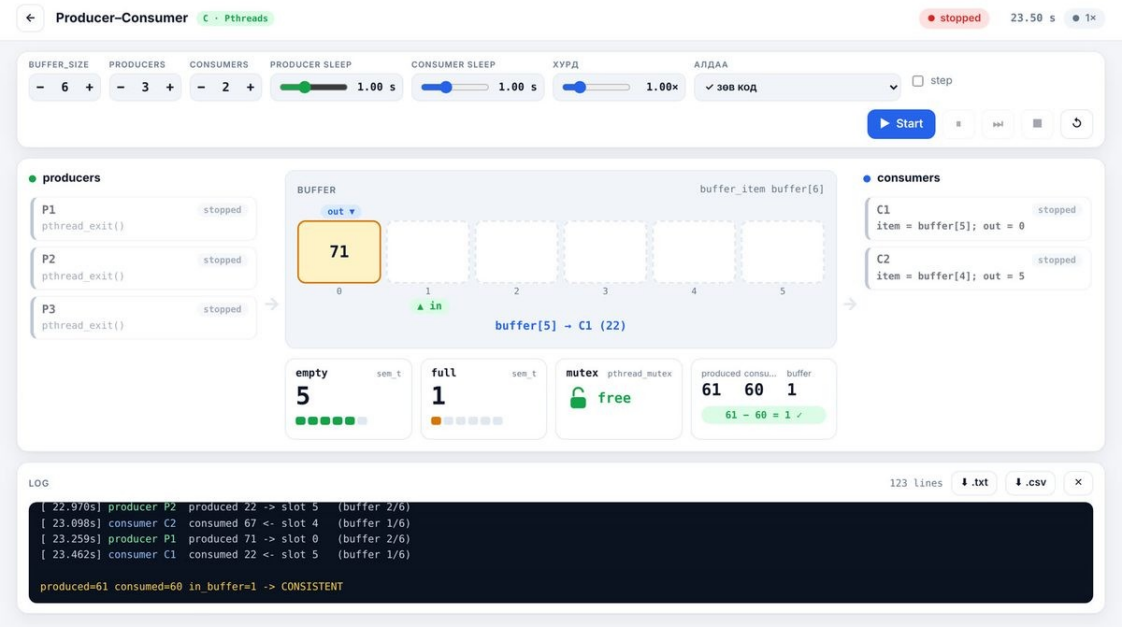
### 6.3 UI-ийн дэлгэцийн зураг



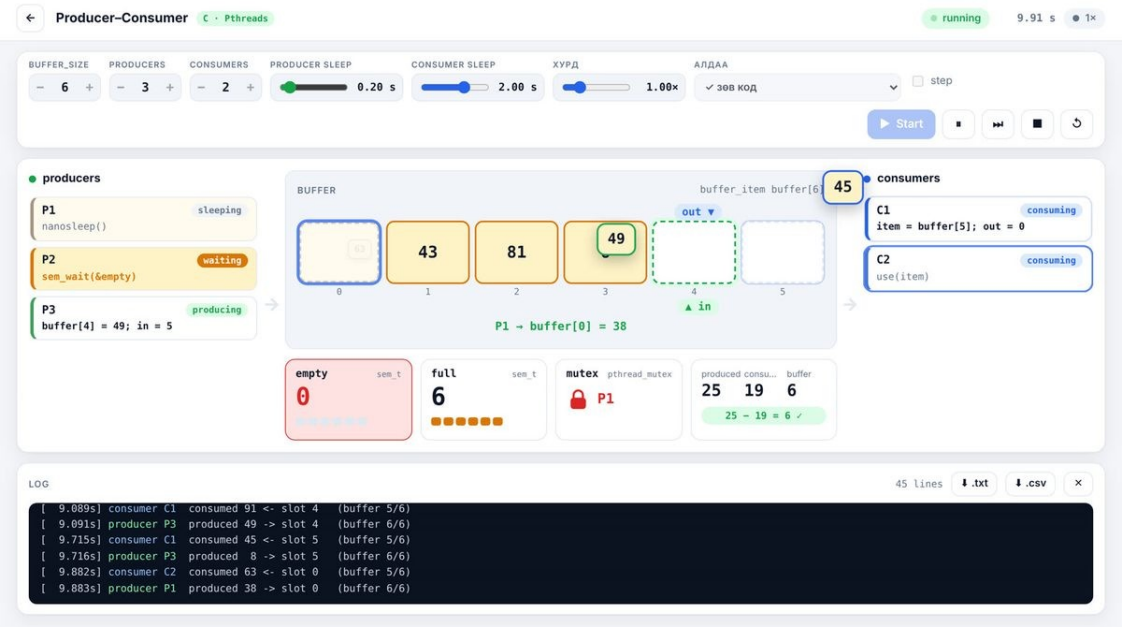

*Зураг 2. Веб UI-ийн нүүр хуудас: «Шуудангийн хайрцаг (Python)» ба «Шууд ажиллагаа (C · Pthreads)» горимыг сонгоно*



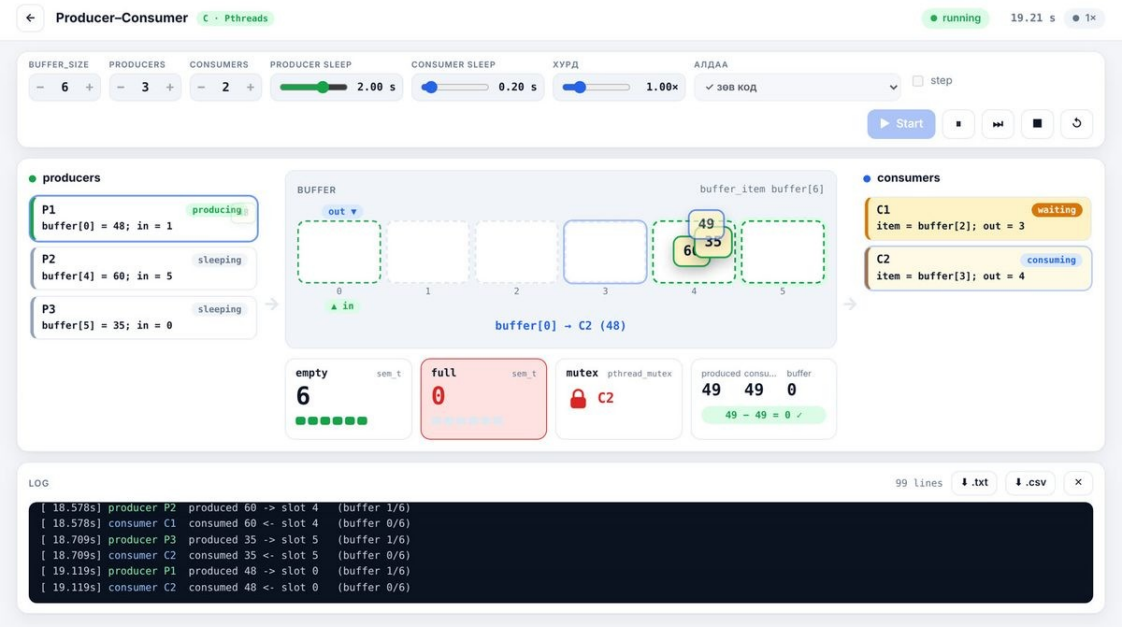

*Зураг 3. C · Pthreads хуудас: ажиллаж байх үе (BUFFER_SIZE = 6, 3 producer, 2 consumer). P2-ийн бичсэн item 94 буферээс C1 рүү нисэж байна*




*Зураг 4. Stop дарсны дараах эцсийн байдал: шалгалт 61 − 60 = 1 (ногоон), лог дээр CONSISTENT*




*Зураг 5. Буфер дүүрсэн (producer sleep 0.2 с, consumer sleep 2 с): empty = 0, P2 нь sem_wait(&empty) дээр хүлээж байна*




*Зураг 6. Буфер хоосорсон (producer sleep 2 с, consumer sleep 0.2 с): full = 0, consumer-ууд sem_wait(&full) дээр хүлээж байна*

# 7. Туршилтын арга

## 7.1 Туршилтын орчин

*Хүснэгт 7.1. Туршилтын орчин*

| Хэсэг: | Утга: |
|---|---|
| Үйлдлийн систем | Ubuntu 24.04 LTS (x86-64, Linux) |
| Хөрвүүлэгч | gcc 13.3.0 (`-Wall -Wextra -O2 -std=c11 -D_DEFAULT_SOURCE -pthread`), санууулгагүй хөрвүүлэгдсэн |
| Build хэрэгсэл | GNU Make 4.3 |
| Python | Python 3.12 (зөвхөн стандарт сан, `unittest`) |
| Бусад хэрэгсэл | ThreadSanitizer (`-fsanitize=thread`) |
| Браузер (UI) | Chrome / Edge / Firefox-ийн сүүлийн хувилбар, 1280×720 ба түүнээс том дэлгэц |

Туршилтыг гурван түвшинд хийсэн: (1) C програмыг командын мөрөөр (`make test`), (2) Python `unittest`-ээр програмын нэгж ба HTTP API-г бодит локал серверээр, (3) вэб UI-ийг хөтөч дээр гараар ажиглаж дэлгэцийн зураг авах.

## 7.2 Туршилтын өгөгдөл

Producer, consumer-ийн тоо, унтах хугацаа, буферийн хэмжээг янзлан өөр өөр ачаалал үүсгэсэн. Мөн алдааны горим болон буруу оролтыг тусад нь туршсан. Командын хэлбэр: `./producer_consumer -q <sleep> <producers> <consumers> [max_nap]` (`-q` нь зөвхөн эцсийн дүнг хэвлэнэ).

*Хүснэгт 7.2. Туршилтын өгөгдлийн жишээ*

| № | Туршилтын төрөл | Параметр | Хүлээгдэж буй үр дүн |
|---|---|---|---|
| T1 | Хамгийн бага: 1 producer, 1 consumer | `-q 2 1 1 0.05` | CONSISTENT, exit 0 |
| T2 | Тэнцүү: 3 producer, 3 consumer | `-q 2 3 3 0.05` | CONSISTENT, exit 0 |
| T3 | Буфер дүүрэх: 8 producer, 1 consumer | `-q 2 8 1 0.05` | CONSISTENT; буферт ≤ 5 item |
| T4 | Буфер хоосрох: 1 producer, 8 consumer | `-q 2 1 8 0.05` | CONSISTENT; буферт ≥ 0 item |
| T5 | Ачаалал: 20 + 20 thread, унтахгүй | `-q 2 20 20 0` | CONSISTENT |
| T6 | Хамгийн их thread: 100 + 100 | `-q 2 100 100 0` | CONSISTENT |
| T7 | Хамгийн бага буфер: BUFFER_SIZE = 1 | `make BUFFER_SIZE=1; -q 2 4 4 0.01` | CONSISTENT; буферт ≤ 1 item |
| T8 | Хамгийн их буфер: BUFFER_SIZE = 20 | `make BUFFER_SIZE=20; -q 2 3 3 0.01` | CONSISTENT; буферт ≤ 20 item |
| E1 | Алдаа: mutex-гүй | `-q --bug no-mutex 1 8 8 0` | INCONSISTENT, lost > 0, exit 2 |
| E2 | Алдаа: `wait(empty)`-гүй | `-q --bug no-empty 1 4 1 0.01` | INCONSISTENT, lost > 0, exit 2 |

| № | Туршилтын төрөл | Параметр | Хүлээгдэж буй үр дүн |
|---|---|---|---|
| E3 | Алдаа: `wait(full)`-гүй | `-q --bug no-full 1 1 4 0.01` | INCONSISTENT, bad_reads > 0, exit 2 |
| E4 | Алдаа: lock → wait дараалал | `-q --bug order 2 2 2 0.05` | DEADLOCK, exit 3 |
| B1–B9 | Буруу оролт | (хоосон), `1 2`, `0 1 1`, `abc 1 1`, `1 0 1`, `1 1 0`, `1 1 101`, `1 1 1 -1`, `1 1 1 x` | Алдааны мэдээлэл, exit 1 |

### 7.3 Хүлээгдэж буй үр дүн

Туршилтаар дараах шалгуурыг баталгаажуулахаар төлөвлөсөн:

1. **Тууштай байдал:** зөв кодын ажиллагаа бүрийн төгсгөлд `produced − consumed = in_buffer`, `lost = 0`, `bad_reads = 0` (CONSISTENT).
2. **Хязгаар:** буферт хэзээ ч BUFFER_SIZE-аас олон item байхгүй; хоосон буферээс унших явдал гарахгүй.
3. **Синхрончлол:** буфер дүүрэн үед producer, хоосон үед consumer хүлээнэ; mutex дотор нэг thread байна.
4. **Алдаа илрүүлэлт:** алдааны горим бүрд тохирох алдаа (lost, bad_reads, DEADLOCK) илэрч, exit код 2 эсвэл 3 болно.
5. **Оролтын шалгалт:** буруу аргументыг хүлээж авахгүй, тодорхой мэдээлэл хэвлэж exit 1 буцаана.
6. **Цэвэр дуусгалт:** Stop дарахад бүх thread (deadlock-той үед ч) `join` хийгдэж, програм гацахгүй.
7. **UI:** Start, Pause, Step, Resume, Stop, Reset ба хурдны тохиргоо зориулалтын дагуу ажиллана.

Хүлээгдэж буй үр дүн нь бодит туршилтаас ялгаатай байж болох тул эцсийн үнэлгээг зөвхөн хийж гүйцэтгэсэн туршилтын мэдээлэлд үндэслэн гаргана. Thread-үүд санамсаргүй хугацаагаар унтдаг тул `produced`, `consumed`-ийн тоо ажиллуулах бүрд өөр байх боловч шалгуур нь тогтмол.

## 8. Үр дүн

Туршилтыг Хүснэгт 7.1-д заасан орчинд, бодит ажиллуулсан гаралтыг тэмдэглэв.

## 8.1 Хэвийн ба хязгаарын тестийн үр дүн

*Хүснэгт 8.1. C програмын тестийн бодит гаралт*

| № | Туршилтын төрөл | Бодит гаралт | Төлөв |
|---|---|---|---|
| T1 | 1 producer, 1 consumer | produced=79 consumed=79 in_buffer=0 → CONSISTENT | **Амжилттай** |
| T2 | 3 producer, 3 consumer | produced=222 consumed=220 in_buffer=2 → CONSISTENT | **Амжилттай** |
| T3 | 8 producer, 1 consumer (дүүрэх) | produced=87 consumed=82 in_buffer=5 → CONSISTENT | **Амжилттай** |
| T4 | 1 producer, 8 consumer (хоосрох) | produced=78 consumed=78 in_buffer=0 → CONSISTENT | **Амжилттай** |
| T5 | 20 + 20 thread, унтахгүй | produced=725588 consumed=725588 in_buffer=0 → CONSISTENT | **Амжилттай** |
| T6 | 100 + 100 thread | produced=654070 consumed=654070 in_buffer=0 → CONSISTENT | **Амжилттай** |
| T7 | BUFFER_SIZE = 1 | produced=1382 consumed=1381 in_buffer=1 → CONSISTENT | **Амжилттай** |
| T8 | BUFFER_SIZE = 20 | produced=1119 consumed=1118 in_buffer=1 → CONSISTENT | **Амжилттай** |

Бүх тохиолдолд `produced − consumed = in_buffer` биелсэн: T3-т буфер дүүрэн (5/5), T4-т хоосон (0), T7-д 1 нүдтэй буфер ч зөв ажилласан. 200 thread (T6) ба секундэд 300 мянга гаруй `insert/remove` (T5, T6) ачааллын үед ч item алдагдаагүй, буфер дүүрэх/хоосрох хязгаар зөрчигдөөгүй. Хэвийн ажиллагааны дэлгэцийн дүн Зураг 4-т харагдана (61 − 60 = 1, CONSISTENT).

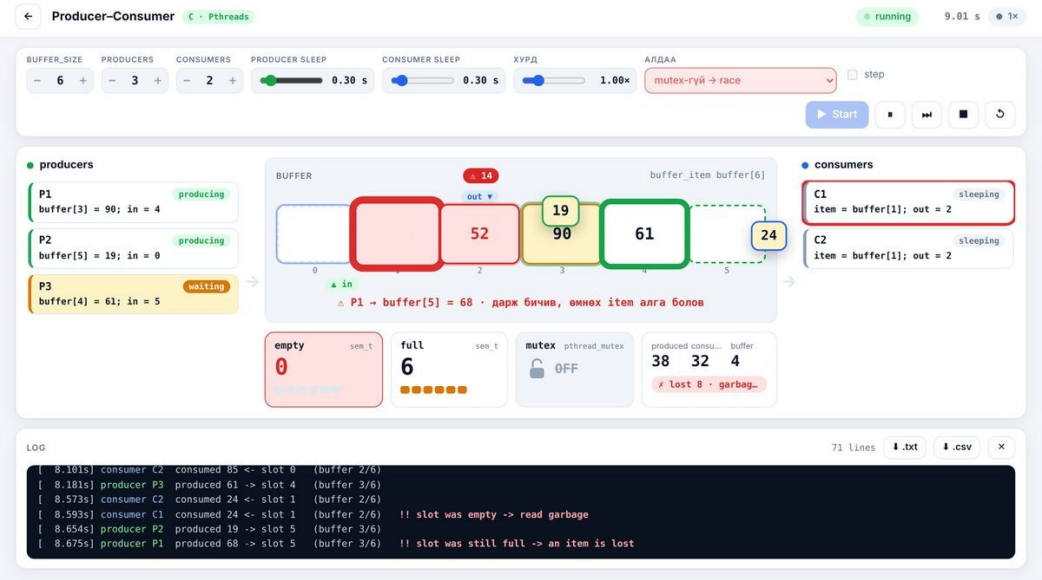
## 8.2 Алдааны горимын туршилтын үр дүн

*Хүснэгт 8.2. Алдааны горимууд (бодит гаралт)*

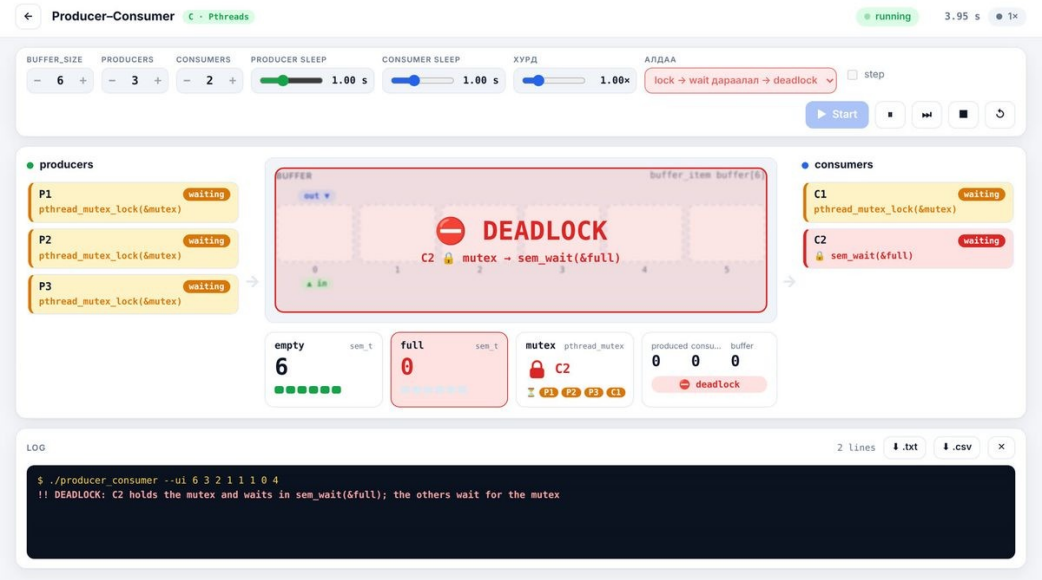
| № | Горим | Бодит гаралт | Exit | Төлөв |
|---|---|---|---|---|
| E1 | `no-mutex` | produced=7013 consumed=7008 in_buffer=4 → INCONSISTENT<br>errors: lost_items=4018 bad_reads=4017 | 2 | **Амжилттай** |
| E2 | `no-empty` | produced=791 consumed=196 in_buffer=5 → INCONSISTENT<br>errors: lost_items=590 bad_reads=0 | 2 | **Амжилттай** |
| E3 | `no-full` | produced=204 consumed=784 in_buffer=0 → INCONSISTENT<br>errors: lost_items=0 bad_reads=580 | 2 | **Амжилттай** |
| E4 | `order` | produced=1 consumed=1 in_buffer=0 → DEADLOCK<br>no progress for the last 2.0s (thread holds the mutex while it waits on a semaphore) | 3 | **Амжилттай** |

Алдааны горим бүрд хүлээгдсэн алдаа яг илэрсэн: **E1** — mutex-гүй үед item мянга мянгаараа алга болж (lost) хоосон нүднээс уншсан (bad_reads); **E2** — producer дүүрэн буферт дарж бичсэн тул уншаагүй item устсан (lost) ч хоосон нүднээс унших явдал гараагүй; **E3** — consumer хоосон нүднээс уншсан (bad_reads), `consumed` нь `produced`-

аас олон болсон нь буфер сөрөг болсныг харуулна; **E4** — бараг эхэлмэгц thread-үүд түгжигдэж, дараагийн 2 секундэд ахиц гараагүй тул DEADLOCK гэж тодорхойлогдож, thread-үүд цэвэрхэн дууссан (програм гацаагүй).




*Зураг 7. Mutex-гүй үед: P1 дүүрэн нүд рүү дарж бичиж, өмнөх item алга болсон (улаан нүд); логонд «!! slot was still full → an item is lost»*




*Зураг 8. Deadlock: C2 нь mutex-ийг барьсаар sem_wait(&full) дээр хүлээж, бусад нь mutex хүлээж байна*



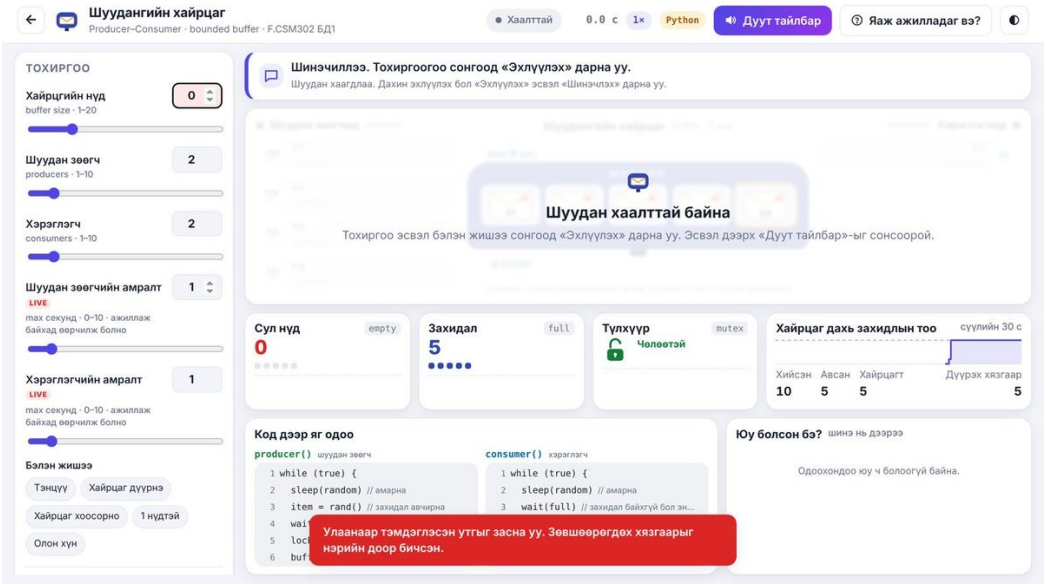

*Зураг 9. Татаж авсан лог: «!!» тэмдэгтэй мөрүүд нь илэрсэн алдаа (lost, garbage)*

## 8.3 Буруу оролтын тестийн үр дүн

*Хүснэгт 8.3. Буруу аргументын шалгалт (exit 1)*

| № | Оролт | Бодит мэдээлэл | Төлөв |
|---|---|---|---|
| B1 | (аргументгүй) | usage: ./producer_consumer [-q] [--bug KIND] <sleep_seconds> <producers> <consumers> [max_nap_seconds] | **Амжилттай** |
| B2 | `1 2` | usage: … (аргумент дутуу) | **Амжилттай** |
| B3 | `0 1 1` | error: sleep_seconds must be a number in (0, 3600], got '0' | **Амжилттай** |
| B4 | `abc 1 1` | error: sleep_seconds must be a number in (0, 3600], got 'abc' | **Амжилттай** |
| B5 | `1 0 1` | error: producers must be an integer 1..100, got '0' | **Амжилттай** |
| B6 | `1 1 0` | error: consumers must be an integer 1..100, got '0' | **Амжилттай** |
| B7 | `1 1 101` | error: consumers must be an integer 1..100, got '101' | **Амжилттай** |
| B8 | `1 1 1 -1` | error: max_nap_seconds must be a number 0..10, got '-1' | **Амжилттай** |
| B9 | `1 1 1 x` | error: max_nap_seconds must be a number 0..10, got 'x' | **Амжилттай** |
| B10 | `--bug x 1 1 1` | error: unknown --bug 'x' | **Амжилттай** |

Бүх тохиолдолд програм thread үүсгэхгүйгээр exit 1-ээр гарсан. Веб UI-д хүрээнээс гарсан утга (жишээ нь BUFFER_SIZE = 0) оруулбал талбар улаан болж, сервер HTTP 400 ба монгол мессеж буцаана (Зураг 10).




*Зураг 10. Буруу оролт: хайрцгийн нүд (BUFFER_SIZE) = 0 үед улаан мэдэгдэл гарсан*

## 8.4 Автомат тестийн үр дүн

*Хүснэгт 8.4. Автомат тестүүд*

| Тест: | Тоо: | Агуулга: | Төлөв |
|---|---|---|---|
| `tests/test_buffer.py` | 12 | FIFO ба тойрог шилжилт, зогсоход -1 буцаах, оролтын шалгалт, олон thread-ийн тууштай байдал (4 төрлийн ачаалал), ажиллаж байх үеийн хурдны тохиргоо | **Амжилттай** |
| `tests/test_c.py` | 6 | C програмын build, CONSISTENT, алдааны 4 горим, буруу аргумент | **Амжилттай** |
| `tests/test_web.py` | 24 (C: 12, Python: 12) | Хуудас, start/pause/step/resume/stop/reset, буруу параметр, үйл явдлын тууштай байдал, deadlock илрүүлэлт | 23 амжилттай, 1 алгассан |
| `make -C src/c test` | — | Хэвийн, алдааны, буруу оролтын CLI тестүүд (8.1–8.3) | ALL TESTS PASSED |
| `make -C src/c tsan` | — | ThreadSanitizer: 16 thread-ийн ачаалал ба `--ui` горимын pause/step/resume/stop | WARNING 0, exit 0 |

`python3 -m unittest discover -s tests` нь **42 тест** ажиллуулж 34 секундэд дууссан: **OK (skipped=1)**. Алгассан нэг тест нь `test_mutex_off` — mutex-ийн унтраалга зөвхөн C хөдөлгүүрт байдаг тул Python хөдөлгүүрийн хувилбарт зориудаар алгасдаг.

## 8.5 Хэрэглэгчийн интерфейсийн (UI) тестийн үр дүн

*Хүснэгт 8.5. UI удирдлагын тест*

| № | Туршилтын төрөл | Хүлээгдэж буй үр дүн | Бодит үр дүн | Төлөв |
|---|---|---|---|---|
| U1 | Start | C процесс эхэлж, thread-үүд ажиллана | `/api/start` → 200; үйл явдал ирсэн (`insert`, `remove` тоо = produced, consumed) | **Амжилттай** |

| № | Туршилтын төрөл | Хүлээгдэж буй үр дүн | Бодит үр дүн | Төлөв |
|---|---|---|---|---|
| U2 | Pause | Thread-үүд зогсоно | `paused = true` төлөв тэмдэглэгдсэн (Pause төлвөөр эхлүүлж шалгасан) | **Амжилттай** |
| U3 | Step | Түр зогссон үед нэг алхам урагшилна | 4 удаа Step дараад produced + consumed нэмэгдсэн | **Амжилттай** |
| U4 | Resume | Ажиллагаа үргэлжилнэ | `paused = false` | **Амжилттай** |
| U5 | Stop | Бүх thread join хийгдэж эцсийн дүн гарна | `stopped = true`; produced − consumed = буферт үлдсэн | **Амжилттай** |
| U6 | Reset | Эхний төлөвт буцна | Төлөв хоосорсон (`sim = null`) | **Амжилттай** |
| U7 | Хурд, унтах хугацаа (ажиллаж байхад) | Шууд хүчин төгөлдөр болно | speed 0.5 → 4, sleep 0.2 / 0.3 тэр даруй шинэчлэгдсэн; хүрээнээс гарсан утгыг 400-аар татгалзсан | **Амжилттай** |
| U8 | Буруу параметр | Монгол алдааны мэдээлэл | size=0, «abc», consumers=99, sleep=-1 → HTTP 400 | **Амжилттай** |
| U9 | Ажиллаж байхад дахин Start | Татгалзана | HTTP 400 | **Амжилттай** |
| U10 | Алдааны горим (mutex-гүй, deadlock, буруу төрөл) | Горим идэвхжиж, deadlock илрэнэ; буруу төрлийг татгалзана | `mutex_on = false`; `deadlock = true` 6 секундийн дотор; буруу төрөл → 400 «Алдааны төрөл…» | **Амжилттай** |

U1–U10-ыг `test_web.py` бодит HTTP дуудлагаар автоматаар шалгасан. Animation-ийн харагдах байдал (нислэг, заагч, өнгө, thread-ийн карт)-ыг 6.3, 8.2-ын дэлгэцийн зургуудаар баталгаажуулсан.

### 8.6 Үр дүнгийн тайлбар

Туршилтын үр дүнгээс харахад зөв код бүх ачаалал, буферийн хэмжээ (1–20), thread-ийн тоо (2–200)-д item алдагдуулалгүй, буферийн хязгаарыг зөрчөлгүй ажилласан. Харин синхрончлолын аль нэг хэсгийг хассан эсвэл дарааллыг буруу болгосон дөрвөн тохиолдол бүрд тохирох алдаа тодорхой илэрсэн нь `empty`, `full`, `mutex` тус бүр өөр өөр үүрэгтэйг баталж байна: `mutex` — хуваалцсан индекс ба тоолуурын бүрэн бүтэн байдал; `empty` — дүүрэн буферт бичихээс хамгаалах; `full` — хоосон буферээс унших явдлаас хамгаалах; `wait`-ийн дараалал — deadlock-оос сэргийлэх. ThreadSanitizer анхааруулга гаргаагүй.

## 9. Хэлэлцүүлэг

### 9.1 Үр дүнгийн тайлбар

Эцсийн шалгалтыг зөвхөн «програм гацаагүй» гэж бус, `produced − consumed = in_buffer` (item алдагдаагүй, давхардаагүй) ба `lost = 0`, `bad_reads = 0` гэсэн нөхцөлөөр хийсэн. Ингэснээр нэг thread-ийн бичсэн item нөгөө thread-д дарагдсан эсэх, хоосон нүднээс уншсан эсэх зэргийг тоолж илрүүлдэг. UI-ийн animation нь бодит C процессын үйл явдал дээр үндэслэсэн тул дэлгэц дээрх thread-ийн хүлээлт, mutex

эзэмшигч нь програмын бодит төлөвтэй таарна. Харагдах байдлыг гараар (дэлгэцийн зургаар), удирдлагын үйлдлийг автоматаар тусад нь шалгасан.

### 9.2 Хязгаарлалт

- C хөдөлгүүр зөвхөн Linux / WSL2 дээр ажиллана (`sem_timedwait`, `pthread_mutex_timedlock`). macOS дээр нэргүй семафор (`sem_init`) дэмжигдэхгүй тул туршаагүй. WSL2-гүй Windows дээр зөвхөн Python горим ажиллана.
- Mutex-ийг микросекунд л барьдаг тул UI дээр сүүлийн эзэмшигчийг ~0.5 секунд харуулдаг (зөвхөн дүрслэлийн сунгалт).
- UI төлөвийг 50–100 мс тутам уншдаг. Нэг удаад ирсэн үйл явдлуудыг дарааллаар нь тоглуулдаг тул animation бодит хугацаанаас бага зэрэг хоцорч, хэт хурдан үед зарим нислэгийг алгасна. Тоолуур ба лог үргэлж бодит утгаа харуулна.
- `empty + full` нь UI-ийн snapshot дээр түр зуур BUFFER_SIZE-аас 1–2-оор бага харагдаж болно (thread `sem_wait`-ийн дараа, `sem_post`-оос өмнө байх үед). Энэ нь алдаа биш.
- `no-mutex` туршилтад race condition-ийг харагдуулахын тулд critical section дотор түр завсар (UI 40 мс, CLI 0.1 мс) нэмсэн. Бодит програмд race condition иймм олон удаа гарахгүй ч боломж нь адилхан.
- ThreadSanitizer нь зөвхөн тухайн ажиллагаанд race илрээгүйг харуулна; race огт байхгүй гэдгийг баталгаажуулахгүй.
- `produced`, `consumed`-ийн тоо ажиллуулах бүрд санамсаргүй тул тайлан дахь тоо яг давтагдахгүй; шалгуур (CONSISTENT) тогтмол.
- Дуут тайлбарын аудиог интернеттэй үед урьдчилан үүсгэх шаардлагатай. Сервер зөвхөн `127.0.0.1` дээр сонсдог.

## 10. Дүгнэлт

Энэхүү төслөөр сурах бичгийн Programming Project 4 буюу Producer–Consumer бодлогыг C хэл дээр POSIX Pthreads, `empty` / `full` тоолох семафор болон `pthread_mutex_t` ашиглан хэрэгжүүлж, thread бүрийн үйлдэл, семафорын утга, mutex-ийн эзэмшлийг buffer animation бүхий веб интерфейсээр харагдуулах зорилгоо биелүүллээ. Бүх синхрончлол C програм дотор бодитоор явагдаж, UI нь зөвхөн харуулж удирдах үүрэгтэй тул animation нь жинхэнэ ажиллагааг тусгана.

Туршилтаар зөв код хэвийн ба хязгаарын (BUFFER_SIZE 1–20, 2–200 thread) бүх тохиолдолд `CONSISTENT` үр дүн гаргаж, алдааны дөрвөн горим (race, overflow, underflow, deadlock) бүрийг илрүүлсэн. 42 автомат тестээс 41 нь амжилттай, 1 нь зориудаар алгассан; ThreadSanitizer анхааруулга гаргаагүй.

Цаашид macOS / Windows (нэргүй семафорын орлуулалт) дэмжих, нэгээс олон буфертай (жишээ нь pipeline) хувилбар нэмэх, мөн Readers–Writers, Dining Philosophers зэрэг бусад синхрончлолын бодлогыг ижил animation-аар харуулах боломжтой.

## 11. Багийн гишүүдийн оролцоо

Гишүүн бүрийн гүйцэтгэсэн ажлыг repository-ийн commit түүх (нийт 23 commit) болон гарын авлагад үндэслэн харууллаа.

*Хүснэгт 11.1. Багийн гишүүдийн оролцоо*

| Гишүүн: | Гүйцэтгэсэн ажил: |
|---|---|
| Н. Баасандорж /B241910012/ (18 commit) | Веб UI (`home.html`, `mail.html`, `raw.html`) ба сервер `src/gui.py`.<br>C / Pthreads хэрэгжүүлэлт `src/c/` ба алдааны туршилтууд (race, overflow, underflow, deadlock).<br>Монгол дуут тайлбар, `tests/test_c.py`, `tests/test_web.py`, README ба гарын авлага. |
| Э. Анунбаяр /B241910035/ (5 commit) | Repository үүсгэж, төслийн анхны бүтцийг (README, `src/`, `tests/`) бий болгосон.<br>Python хэрэгжүүлэлт `src/bounded_buffer.py`: `validate_params`, `BoundedBuffer`, `Worker`, `Simulation`, номын хэлбэрийн CLI.<br>`tests/test_buffer.py`. |
| B222230020 ..... | ..... |
| B242270014 ..... | ..... |
| B232270028 ..... | ..... |

## Ашигласан материал

- A. Silberschatz, P. B. Galvin, G. Gagne. Operating System Concepts, 10th ed., Wiley, 2018. 7.1.1 (The Bounded-Buffer Problem, Figures 7.1, 7.2); Programming Project 4: The Producer–Consumer Problem.
- F.CSM302 Үйлдлийн систем — Бие даалт 1-ийн удирдамж.
- POSIX man хуудас: `pthread_create(3)`, `pthread_mutex_timedlock(3)`, `sem_init(3)`, `sem_timedwait(3)`.
- Төслийн репозитори: github.com/anunbayar1/CSM302-2627-BD1 — `README.md`, `src/`, `tests/`, `docs/V-1-12_Manual.pdf`.
- Дуут тайлбар: Microsoft Edge TTS-ийн монгол хоолой (`edge-tts`).
- AI ашиглалт: Claude (Anthropic)-г код, тест, UI боловсруулахад (README-д дурдсан ёсоор) болон энэ тайлангийн төсөл бэлтгэхэд ашигласан.